Class Imbalance in the BDD100K dataset.

The BDD100K dataset is imbalanced. There is more cars than any other object in the dataset. Take a look at these graphs:

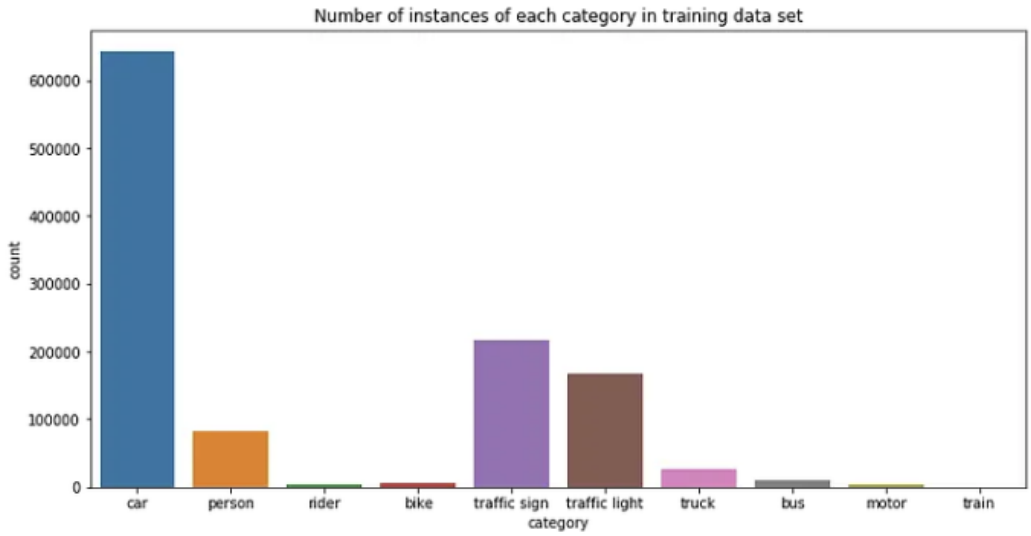
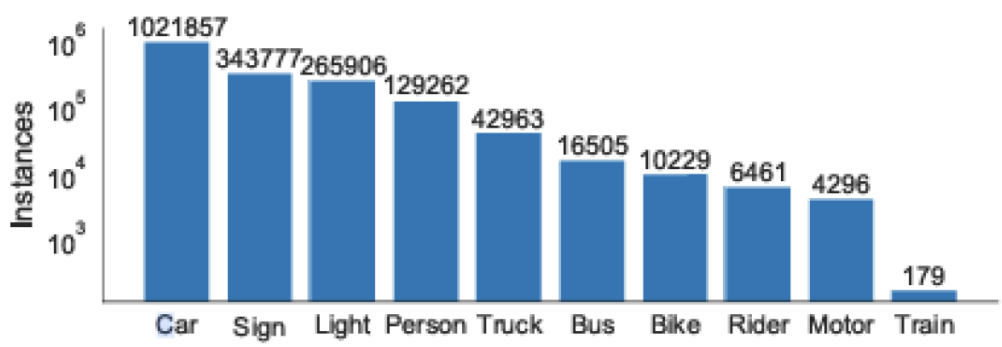


Figure 8 Number of instances of each category



(a) Number of instances in each category.

Cars are clearly the class that is most represented.

This explains why my model has only been able to detect cars consistently and it explains why it has struggled detecting objects like buses, trucks, bikes, riders, and trains. It does an okay job (not great) at detecting people, traffic signs, and traffic lights.

Solutions:

1. **Loss weighting**
   Give more importance to rare classes.
   Idea:
   Instead of treating all classes equally: loss = classification_loss
   Use weighted loss: **loss = weight[class] * classification_loss**
   Example weights (inverse frequency):
   - car → 1
   - person → 2
   - bike → 10
   - motor → 20
   - train → 50

   👉 Rare classes get stronger gradients
2. **Focal Loss**
   This is used in models like RetinaNet.
   It focuses on:

   - Hard examples
   - Rare classes

   Instead of standard cross entropy:

   👉 Use:

   - **Focal Loss (γ ≈ 2, α balancing)**

   This:

   - Reduces impact of easy (car) examples
   - Boosts learning for hard (bike/motor)

3. **Oversampling rare classes**
   Make the model *see rare classes more often*
   Options:
   - Duplicate images with bikes/motors
   - Sample dataset like:
     - 50% normal
     - 50% rare-class-heavy images

👉 This is VERY effective in practice

Understanding Loss weighting (ChatGPT explanation)

🧠 **Step 1: What problem are we solving?**

Your dataset looks like this (simplified):

Class	Count
car	1,000,000
bike	10,000
motor	4,000

❗ **What happens during training?**

Your model learns by minimizing **loss**.

Each mistake contributes to the total loss.

So naturally:

- Many car examples → lots of car-related loss
- Few motor examples → very little motor-related loss

⚠️ **Result**

The model learns:

"If I get cars right, I'll reduce most of the loss"

So, it **ignores rare classes**.

💡 **Step 2: What is loss weighting?**

Loss weighting simply says:

"Not all mistakes are equally important"

We manually adjust importance.

🎯 Analogy

Imagine a teacher grading:

- Car questions = 1 point
- Motor questions = 20 points

Now students (your model) will:

- Care much more about motor questions

🔥 In ML terms

Instead of:

```
total_loss = car_loss + motor_loss
```

We do:

```
total_loss = 1 * car_loss + 20 * motor_loss
```

👉 Now motor mistakes matter much more

🧠 Step 3: How does this look in your model?

Your model outputs:

```
[N, num_classes]
```

And you already compute:

```
loss_classification = sigmoid_focal_loss(...)
```

🔑 Key idea

Right now:

- Every class contributes equally

We want:

- Each class has its own importance (weight)

🧪 **Step 4: Visual intuition**

Let's say for one prediction:

car loss = 0.2
motor loss = 0.2

Without weighting:

total = 0.4

After weighting

car weight = 1
motor weight = 10

Now:

total = (1 × 0.2) + (10 × 0.2)
= 0.2 + 2.0
= 2.2

👉 Motor dominates learning now

🧠 **Step 5: Why this works**

During backpropagation:

- Bigger loss → bigger gradients
- Bigger gradients → stronger learning

So:

Rare classes now "push" the model harder

⚙️ **Step 6: Where you apply it (important)**

You already have:

loss_classification = sigmoid_focal_loss(..., reduction="sum")

🚨 **Problem**

When you use "sum":

- Everything is already combined
- You can't adjust per class anymore

✅ Fix

You need:

```
reduction="none"
```

So you get:

```
loss shape = [N, num_classes]
```

Now you can:

- Multiply each class by its weight

🧠 Step 7: Conceptual flow

Here's the full idea:

1. Compute loss per class (no reduction)
2. Multiply each class by its weight
3. Sum everything

🧪 Step 8: What are "class weights"?

A simple list:

[car, person, bike, motor, ...]

Example:

car → 1
person → 2
bike → 10
motor → 15
train → 30

🧠 How do we choose them?

Basic rule:

The rarer the class → the bigger the weight

⚠️ Step 9: Important cautions

❌ **Too large weights**

- Training becomes unstable
- Loss explodes

❌ **Too small weights**

- No real effect

✅ **Good approach**

- Start moderate
- Adjust after observing results

🔍 **Step 10: What you should expect**

After adding loss weighting:

- Rare classes start appearing
- AP for bike/motor increases
- Car AP might drop slightly (normal)

🧠 **Your question (reframed)**

"If there are very few examples, how can the model learn those classes even if we increase their importance?"

👉 Short answer:

Loss weighting doesn't create new information — it just makes better use of the little information you have.

🔍 **Step 1: What learning actually needs**

For a model to learn a class, it needs:

1. **Signal** → examples of that class
2. **Attention** → gradients that reinforce it

❗ Your current situation

You *do* have signal:

- Bikes exist
- Motors exist
- Riders exist

But:

The model is basically ignoring them because their signal is too weak

💡 Step 2: What loss weighting really does

Loss weighting says:

"When you DO see a rare class, take it VERY seriously"

Analogy (this will click)

Imagine:

- You only get **3 questions about motorcycles** on an exam
- But each is worth **50 points**

👉 You will:

- Study those 3 questions VERY carefully
- Extract as much pattern as possible

In your model

Even if:

- motor appears only 4,000 times

With weighting:

- each of those 4,000 examples becomes **high-impact**

🔥 **Step 3: Why this actually works**

Neural networks don't just memorize — they **generalize patterns**

Even a small number of examples can teach:

- shape (thin vs wide)
- structure (two wheels)
- context (on road, near riders)

Example

From a few bike images, the model learns:

- circular shapes → wheels
- narrow structure
- typical positioning

👉 Then it applies that to unseen images

⚠️ **Step 4: The limitation (very important)**

You're absolutely right to be skeptical.

❗ If data is TOO scarce → weighting alone is NOT enough

Example: your "train" class

- Only ~179 instances

👉 That's borderline:

- Model may not learn it well at all
- Even with weighting

🧠 **Step 5: So what actually helps?**

Think of it like a combination:

🥇 Loss weighting

✓ Makes rare examples matter more
✓ Fixes gradient imbalance

🥈 Seeing them more often (sampling)

✓ Helps reinforce patterns
✓ Prevents forgetting

🥉 Data augmentation

✓ “Artificially” increases data
✓ Adds variation

🧠 Combined effect

Even with few examples:

More importance + more exposure + more variation
→ Enough signal to learn

🔥 Key insight (this is the answer to your question)

Loss weighting doesn’t *create* learning — it **unlocks learning that was already possible but suppressed**

🧪 Concrete example from your model

Right now:

- bike AP ≈ 0.0099 → almost ignored

After weighting:

- same data
- BUT stronger gradients

👉 model starts detecting bikes

⚖️ **Honest reality check**

- For **bike/motor/rider** → weighting will help a lot
- For **train** → you may still struggle

That's normal.

🚀 **What this means for YOU**

You are not trying to:

"make the model learn from nothing"

You are trying to:

"make the model STOP ignoring rare information"

🧠 **Final mental model**

Think of your dataset as a classroom:

- Cars = loud students
- Bikes = quiet students

Right now:

The teacher only listens to loud students

Loss weighting:

Forces the teacher to listen to the quiet ones too